SchroedingerEquation1D

April 1, 2022

1 Schroedinger Equations in 1D

A quantum mechanical particle under a one dimensional static potetial is described by the **time-dependent Schroedinger Equation**,

$$i\hbar\frac{\partial\psi(x,t)}{\partial t} = \left[-\frac{\hbar^2}{2m}\frac{\partial^2}{\partial x^2} + V(x)\right]\psi(x,t) \tag{1}$$

where t is time, x is the position of the particle, $\psi(x,t)$ is the wavefunction, and $V(x)$ is the potential. m and $\hbar$ are the mass of the particle and Plank's constant, respectively.

Solving the problems of quantum mechanics is equivalent to solving the above partial differential equation. However, it is impossible to analytically solve the solution for a general potential $V(x)$. Hence, we rely on numerical calculations to obtain solutions for the Schroedinger equation.

1.1 Part 1. Discritizing the Continuous Systems (Conversion to Lattice Systems)

In order to solve the continuous systems with computers, we first need to spatially discretize them. Given a continuous function $f(x)$, $f(x_0+\Delta x)$ for $\Delta x \ll 1$ can be expressed using Taylor expansion,

$$f(x_0+\Delta x) = \sum_{n=0}^{\infty}\frac{(\Delta x)^n}{n!}\left.\frac{\partial^n f(x)}{\partial x^m}\right|_{x_0} = f(x_0)+f'(x_0)\Delta x+f''(x_0)\frac{(\Delta x)^2}{2!}+f'''(x_0)\frac{(\Delta x)^3}{3!}+\cdots. \tag{2}$$

Similarly, $f(x_0-\Delta x)$ is expressed as

$$f(x_0-\Delta x) = \sum_{n=0}^{\infty}\frac{(-\Delta x)^n}{n!}\left.\frac{\partial^n f(x)}{\partial x^m}\right|_{x_0} = f(x_0)-f'(x_0)\Delta x+f''(x_0)\frac{(\Delta x)^2}{2!}-f'''(x_0)\frac{(\Delta x)^3}{3!}+\cdots. \tag{3}$$

By adding the two equations above, the first derivative $f'(x)$ and the second derivative f'' are approximated as

$$f'(x_0) \simeq \frac{f(x_0+\Delta x)-f(x_0-\Delta x)}{2\Delta x} + \mathcal{O}((\Delta x)^2), \tag{4}$$

and

$$f''(x_0) \simeq \frac{f(x_0+\Delta x)+f(x_0-\Delta x)-2f(x_0)}{(\Delta x)^2} + \mathcal{O}((\Delta x)^2). \tag{5}$$

Therefore, the Schroedinger Equation is rewritten as

$$i\hbar\frac{\partial\psi(x,t)}{\partial t} = -\frac{\hbar^2}{2m}\frac{\psi(x+\Delta x,t)+\psi(x-\Delta x,t)-2\psi(x,t)}{(\Delta x^2)} + V(x)\psi(x,t). \tag{6}$$

For simplicity, let $\Delta x = 1$, $2m = 1$, and $\hbar = 1$. This is equivalent to using the atomic unit.

Then, we have

$$i\hbar\frac{\partial\psi(x,t)}{\partial t} = [2+V(x)]\psi(x,t) - \psi(x+1,t) - \psi(x-1,t). \tag{7}$$

This is equivalent to the **tight-binding model** commonly used in condensed matter physics.

Let us label each of discretized position (site) x with index from 1 to L.

In addition, let us impose the **fixed end condition**, i.e. the particle at the leftmost(rightmost) site cannot move further to the left(right).

1.2 Part 2. The Time-independent Schroedinger Equation

A **steady state** is a state without change in time. The wavefunction $\psi(x,t)$ of a particle in a steady state can be seperated into the spatial and temporal components. Assuming that a particle in a steady state has the energy $E = \hbar\omega$, the wavefunction is expressed as

$$\psi(x,t) = e^{-iEt/\hbar}\psi(x). \tag{8}$$

This indicates that the wavefunction oscillates as $e^{-iEt/\hbar}$ but the probability amplitude $|\psi(x,t)|^2$ is constant in time.

By substituting Eq.(8) into Eq.(7), the time-independent Schroedinger equation

$$E\psi(x) = [2+V(x)]\psi(x) - \psi(x+1) - \psi(x-1) \tag{9}$$

is obtained.

Let $x \in \{1,2,3,\ldots,M\}$. Then $\psi(x)$ can be considered a vector with M components. Then, rewrite Eq.(9) in a matrix form:

$$E\begin{pmatrix}\psi(1)\\ \psi(2)\\ \psi(3)\\ \vdots\\ \psi(M)\end{pmatrix} = \begin{pmatrix}2+V(1) & -1 & 0 & \cdots & 0\\ -1 & 2+V(2) & -1 & \cdots & 0\\ 0 & -1 & 2+V(3) & \cdots & 0\\ \vdots & \vdots & \vdots & \ddots & \vdots\\ 0 & 0 & 0 & \vdots & 2+V(M)\end{pmatrix}\begin{pmatrix}\psi(1)\\ \psi(2)\\ \psi(3)\\ \vdots\\ \psi(M)\end{pmatrix} = \mathcal{H}\begin{pmatrix}\psi(1)\\ \psi(2)\\ \psi(3)\\ \vdots\\ \psi(M)\end{pmatrix} \tag{10}$$

By solving the eigenvalue problem for the matrix $\mathcal{H}$, the eigen energy E and the eigen wavefunction $\psi(x)$ of the Schroedinger equation for the steady-state lattice system are determined. Since the matrix is $M \times M$, there exists M eigenvalues and M eigenvectors. Let E_m be the m-th eigen energy. Then, there exists the eigen wavefunction

$$\Psi_m(x) = \begin{pmatrix} \psi_m(1) \\ \psi_m(2) \\ \psi_m(3) \\ \vdots \\ \psi_m(M) \end{pmatrix} \tag{11}$$

corresponding to E_m.

Explicitly writing m, the time-independent Schroedinger equation is given by

$$\mathcal{H}\Psi_m = E_m \Psi_m. \tag{12}$$

Finally, the time-dependent eigen state is written as

$$\psi_m(x,t) = e^{-iE_m t/\hbar}\Psi_m(x). \tag{13}$$

1.3 Code:

Import libraries

[1]:
```
using LinearAlgebra  # call the library for linear algebra
using Plots          # call the library for plotting
```

Set the number of sites

[2]:
```
# Number of sites
N = 1000
# Unit length of the systems
a = 0.01
```

[2]: 0.01

Define the Hamiltonian matrix $\mathcal{H}$ for $V(x) = 0$

[3]:
```
# a function to define a Hamiltonian matrix with potential V (by default, V=0)
function make_Hamiltonian(N, V=zeros(Float64, N))
    H = zeros(Float64, N, N)
    for i in 1:N
        H[i, i] = 2.0 / a^2 - V[i]
    end
    for i in 1:N-1
        H[i, i+1] = -1.0 / a^2
        H[i+1, i] = -1.0 / a^2
```

```
    end
    return H
end

H_mat = make_Hamiltonian(N)
```

[3]:
```
1000×1000 Matrix{Float64}:
  20000.0  -10000.0       0.0       0.0  …       0.0       0.0       0.0
 -10000.0   20000.0  -10000.0       0.0          0.0       0.0       0.0
      0.0  -10000.0   20000.0  -10000.0          0.0       0.0       0.0
      0.0       0.0  -10000.0   20000.0          0.0       0.0       0.0
      0.0       0.0       0.0  -10000.0          0.0       0.0       0.0
      0.0       0.0       0.0       0.0  …       0.0       0.0       0.0
      0.0       0.0       0.0       0.0          0.0       0.0       0.0
      0.0       0.0       0.0       0.0          0.0       0.0       0.0
      0.0       0.0       0.0       0.0          0.0       0.0       0.0
      0.0       0.0       0.0       0.0          0.0       0.0       0.0
      0.0       0.0       0.0       0.0  …       0.0       0.0       0.0
      0.0       0.0       0.0       0.0          0.0       0.0       0.0
      0.0       0.0       0.0       0.0          0.0       0.0       0.0

      0.0       0.0       0.0       0.0          0.0       0.0       0.0
      0.0       0.0       0.0       0.0          0.0       0.0       0.0
      0.0       0.0       0.0       0.0  …       0.0       0.0       0.0
      0.0       0.0       0.0       0.0          0.0       0.0       0.0
      0.0       0.0       0.0       0.0          0.0       0.0       0.0
      0.0       0.0       0.0       0.0          0.0       0.0       0.0
      0.0       0.0       0.0       0.0          0.0       0.0       0.0
      0.0       0.0       0.0       0.0  …       0.0       0.0       0.0
      0.0       0.0       0.0       0.0     -10000.0       0.0       0.0
      0.0       0.0       0.0       0.0      20000.0  -10000.0       0.0
      0.0       0.0       0.0       0.0     -10000.0   20000.0  -10000.0
      0.0       0.0       0.0       0.0          0.0  -10000.0   20000.0
```

Solve the eigen values and eigen vectors of $\mathcal{H}$

[4]:
```
,   = eigen(H_mat)
```

[4]:
```
Eigen{Float64, Float64, Matrix{Float64}, Vector{Float64}}
values:
1000-element Vector{Float64}:
     0.09849886676979276
     0.3939944968651252
     0.8864839796963152
     1.575962464288196
     2.462423159364437
     3.545857333382498
```

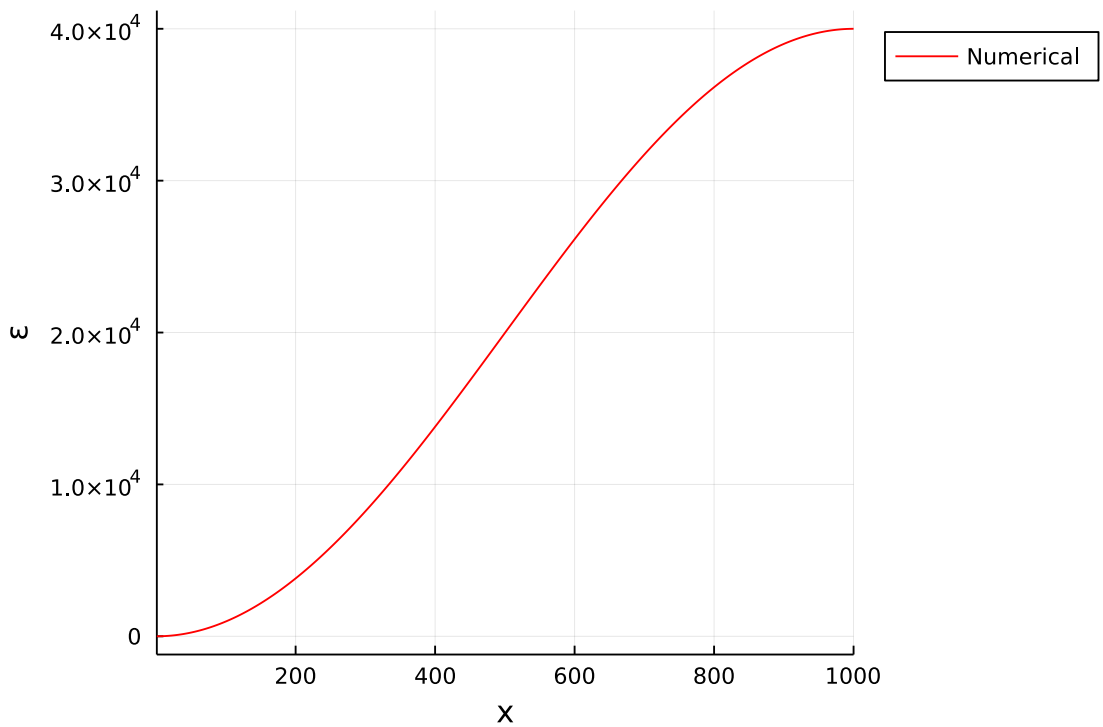
```
     4.826254314643498
     6.303601491374821
     7.977884311881861
     9.849086284660045
    11.917188978596505
    14.182172023099547
    16.644013108346893

 39985.817827976905
 39988.082811021406
 39990.15091371534
 39992.02211568812
 39993.696398508626
 39995.17374568536
 39996.45414266662
 39997.53757684064
 39998.42403753572
 39999.11351602031
 39999.606005503134
 39999.90150113323
vectors:
1000×1000 Matrix{Float64}:
 0.000140286  -0.00028057   0.000420851  …  -0.00028057    0.000140286
 0.00028057   -0.000561129  0.000841665      0.000561129  -0.00028057
 0.000420851  -0.000841665  0.0012624       -0.000841665   0.000420851
 0.000561129  -0.00112217   0.00168303       0.00112217   -0.000561129
 0.0007014    -0.00140263   0.00210351      -0.00140263    0.0007014
 0.000841665  -0.00168303   0.0025238    …   0.00168303   -0.000841665
 0.000981922  -0.00196337   0.00294387      -0.00196337    0.000981922
 0.00112217   -0.00224363   0.00336368       0.00224363   -0.00112217
 0.0012624    -0.0025238    0.00378319      -0.0025238     0.0012624
 0.00140263   -0.00280387   0.00420236       0.00280387   -0.00140263
 0.00154284   -0.00308384   0.00462116   …  -0.00308384    0.00154284
 0.00168303   -0.00336368   0.00503955       0.00336368   -0.00168303
 0.00182321   -0.00364338   0.0054575       -0.00364338    0.00182321

 0.00168303    0.00336368   0.00503955       0.00336368    0.00168303
 0.00154284    0.00308384   0.00462116      -0.00308384   -0.00154284
 0.00140263    0.00280387   0.00420236   …   0.00280387    0.00140263
 0.0012624     0.0025238    0.00378319      -0.0025238    -0.0012624
 0.00112217    0.00224363   0.00336368       0.00224363    0.00112217
 0.000981922   0.00196337   0.00294387      -0.00196337   -0.000981922
 0.000841665   0.00168303   0.0025238        0.00168303    0.000841665
 0.0007014     0.00140263   0.00210351   …  -0.00140263   -0.0007014
 0.000561129   0.00112217   0.00168303       0.00112217    0.000561129
 0.000420851   0.000841665  0.0012624       -0.000841665  -0.000420851
 0.00028057    0.000561129  0.000841665      0.000561129   0.00028057
```

```
0.000140286    0.00028057    0.000420851      -0.00028057    -0.000140286
```

Plot the eigen energy

[5]:
```
plot( , xaxis=("x", (1,N)), yaxis=(" "), label="Numerical", line=(:red), legend=:
↪outertopright)
```

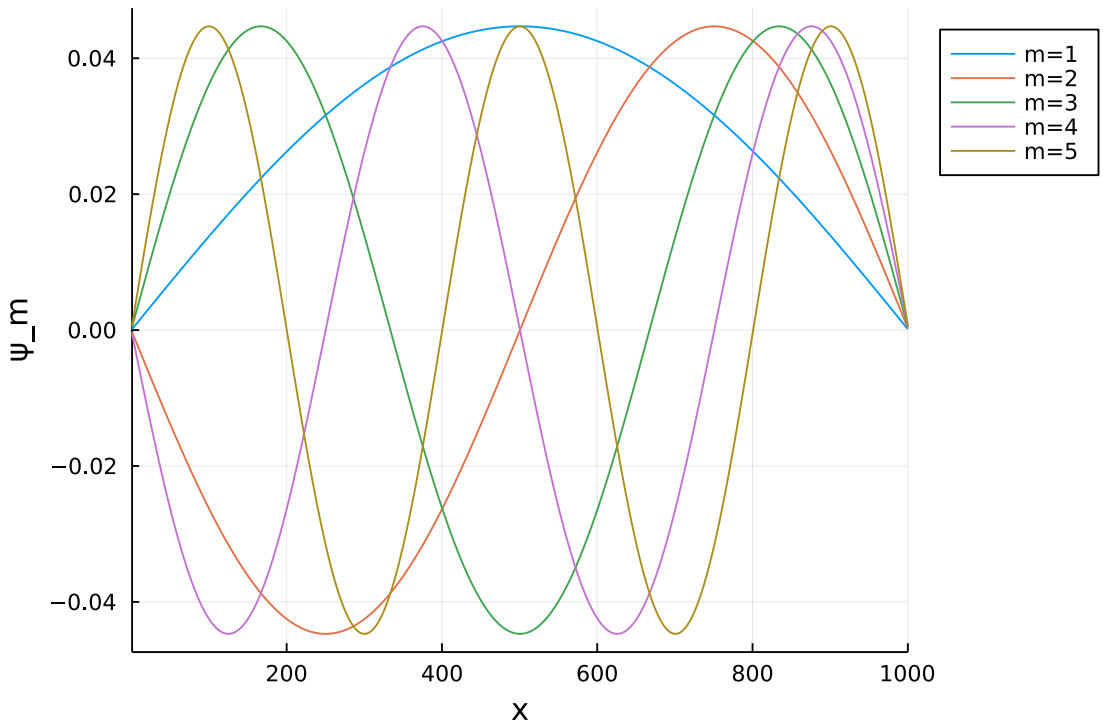
[5]:



Plot the eigen wavefunction

[6]:
```
plot( [1:N, 1:5], xaxis=("x", (1,N)), yaxis=(" _m"), label=["m=1" "m=2" "m=3"␣
↪"m=4" "m=5"], legend=:outertopright)
```

[6]:



In the case of $V(x) = 0$, the Schroedinger equation can be solved analytically.

$$-\frac{\hbar^2}{2m}\frac{d\psi(x)}{dx^2} = \varepsilon\psi(x) \tag{14}$$

The general solution to this ODE is given by

$$\psi(x) = C_1 \exp\left[i\frac{\sqrt{2m\varepsilon}}{\hbar}x\right] + C_2 \exp\left[-i\frac{\sqrt{2m\varepsilon}}{\hbar}x\right] \tag{15}$$

Under the fixed-end condition, $\psi(x = 1) = \psi(x = N) = 0$, the coefficients C_1 and C_2 is determined; $C_1 = -C_2$. Thus, the solution is determined as

$$\psi(x) = 2iC_1 \sin\frac{\sqrt{2m\varepsilon}}{\hbar}x. \tag{16}$$

where

$$\frac{\sqrt{2m\varepsilon}}{\hbar}N = n\pi \quad \forall\, n \in \mathbb{N}. \tag{17}$$

Therefore, the eigen energy must be quantized,

$$\varepsilon = n^2 \frac{\hbar^2 \pi^2}{2mN^2}. \tag{18}$$

[7]:
```
=zeros(Float64, N)
for n in 1:N
    [n] = n^2 *  ^2 / (a * N)^2
end

 = zeros(Float64, N)
for i in 1:N
    [i] = sin(i* /(a * N))
end
 =   / sqrt(sum(dot( ,  )))
```

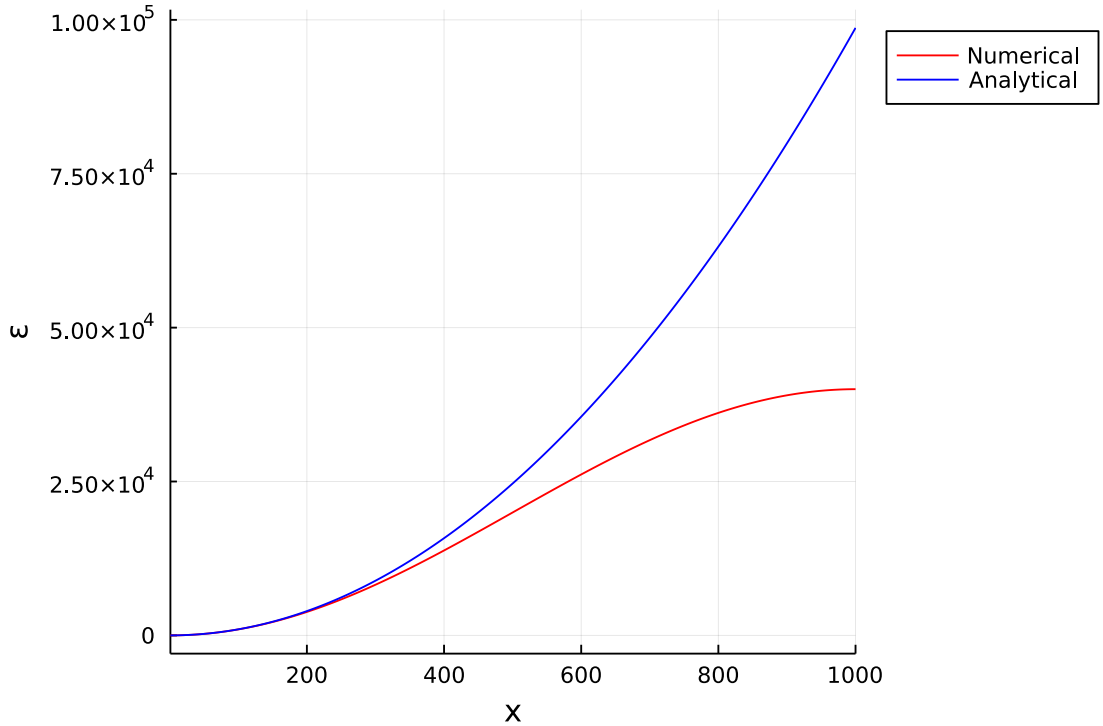
[7]:
```
1000-element Vector{Float64}:
  0.013819660112501051
  0.026286555605956684
  0.036180339887498955
  0.042532540417602
  0.0447213595499958
  0.042532540417602
  0.036180339887498955
  0.026286555605956687
  0.013819660112501056
  5.476786982642027e-18
 -0.01381966011250103
 -0.02628655560595668
 -0.03618033988749895

  0.013819660112502399
 -5.699733893732177e-16
 -0.013819660112501067
 -0.026286555605956243
 -0.0361803398874998
 -0.04253254041760227
 -0.0447213595499958
 -0.04253254041760207
 -0.03618033988749941
 -0.026286555605957766
 -0.013819660112502857
 -2.4542649279581086e-15
```

Let us plot the analytical eigen energy together with the numerical one.

For larger x, the numerical result deviates from the analytical one. This is due to the discretiziation, as the scale shorter than 1 cannot be described in the finite difference method.

[8]:
```
plot([ ,  ], xaxis=("x", (1,N)), yaxis=(" "), label=["Numerical" "Analytical"],␣
↪line=([:red :blue]), legend=:outertopright)
```

[8]:

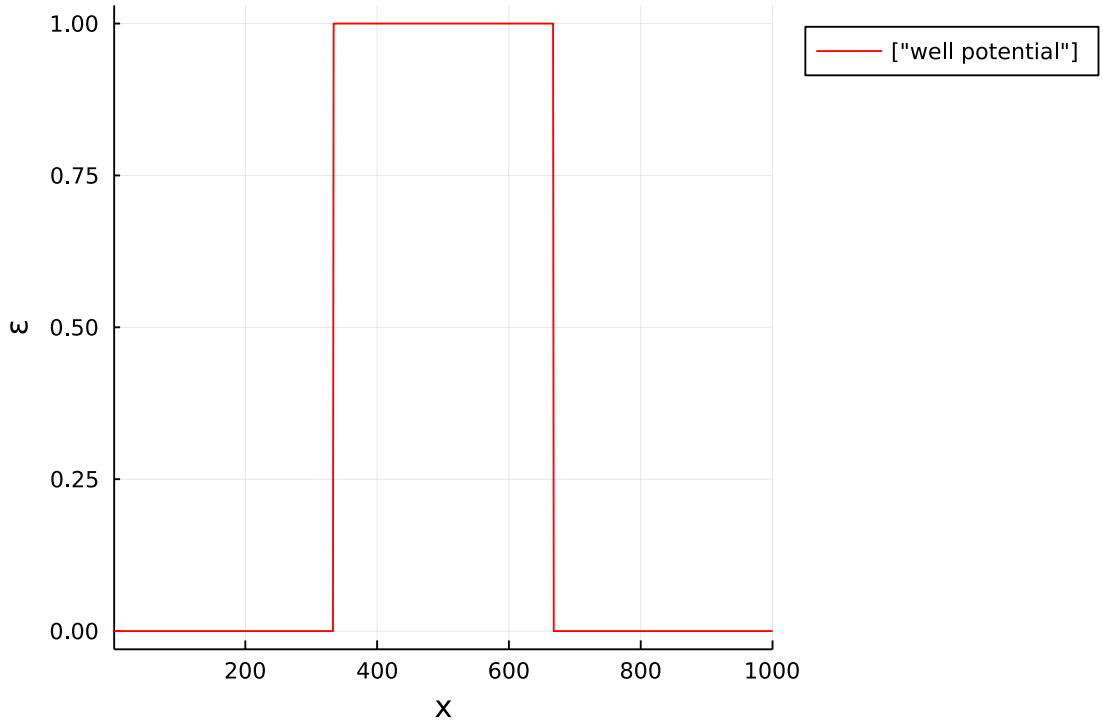


Now, let us consider the following potential:

$$V_{\text{well}}(x) = \begin{cases} V_0 & \text{if } \frac{N}{3} \leq x \leq \frac{2N}{3} \\ 0 & \text{otherwise} \end{cases} \tag{19}$$

[9]:
```
function calc_V(N,V0)
    vec_V = zeros(Float64 ,N)
    dx = N/6
    for i in 1:N
        if (N+1)/2 - dx <= i <= (N+1)/2 + dx
            vec_V[i] = V0
        end
    end
    return vec_V
end
V0=1.0
vec_V = calc_V(N,V0)
plot(vec_V,  xaxis=("x", (1,N)), yaxis=(" "), label=["well potential"], line=(:
↪red), legend=:outertopright)
```

[9]:



[10]:
```
function make_H1dv(N,a,V0)
    mat_H = zeros(Float64,N,N)
    vec_V = calc_V(N,V0)
    for i in 1:N
        for dx in -1:1
            j = i + dx
            v = 0.0
            if dx == 0
                v = (2/a^2 + vec_V[i])
            elseif dx == 1
                v = -1/a^2
            elseif dx == -1
                v = -1/a^2
            end
            if 1 <= j <= N
                mat_H[i,j] = v
            end
        end
    end
    return mat_H
end
```

[10]: make_H1dv (generic function with 1 method)

[11]:
```
N = 1000
a = 0.01
V0 = 1.0
H_well = make_H1dv(N,a,V0)
```

[11]:
```
1000×1000 Matrix{Float64}:
  20000.0  -10000.0       0.0       0.0  …       0.0       0.0       0.0
 -10000.0   20000.0  -10000.0       0.0          0.0       0.0       0.0
      0.0  -10000.0   20000.0  -10000.0          0.0       0.0       0.0
      0.0       0.0  -10000.0   20000.0          0.0       0.0       0.0
      0.0       0.0       0.0  -10000.0          0.0       0.0       0.0
      0.0       0.0       0.0       0.0  …       0.0       0.0       0.0
      0.0       0.0       0.0       0.0          0.0       0.0       0.0
      0.0       0.0       0.0       0.0          0.0       0.0       0.0
      0.0       0.0       0.0       0.0          0.0       0.0       0.0
      0.0       0.0       0.0       0.0          0.0       0.0       0.0
      0.0       0.0       0.0       0.0  …       0.0       0.0       0.0
      0.0       0.0       0.0       0.0          0.0       0.0       0.0
      0.0       0.0       0.0       0.0          0.0       0.0       0.0

      0.0       0.0       0.0       0.0          0.0       0.0       0.0
      0.0       0.0       0.0       0.0          0.0       0.0       0.0
      0.0       0.0       0.0       0.0  …       0.0       0.0       0.0
      0.0       0.0       0.0       0.0          0.0       0.0       0.0
      0.0       0.0       0.0       0.0          0.0       0.0       0.0
      0.0       0.0       0.0       0.0          0.0       0.0       0.0
      0.0       0.0       0.0       0.0          0.0       0.0       0.0
      0.0       0.0       0.0       0.0  …       0.0       0.0       0.0
      0.0       0.0       0.0       0.0     -10000.0       0.0       0.0
      0.0       0.0       0.0       0.0      20000.0  -10000.0       0.0
      0.0       0.0       0.0       0.0     -10000.0   20000.0  -10000.0
      0.0       0.0       0.0       0.0          0.0  -10000.0   20000.0
```

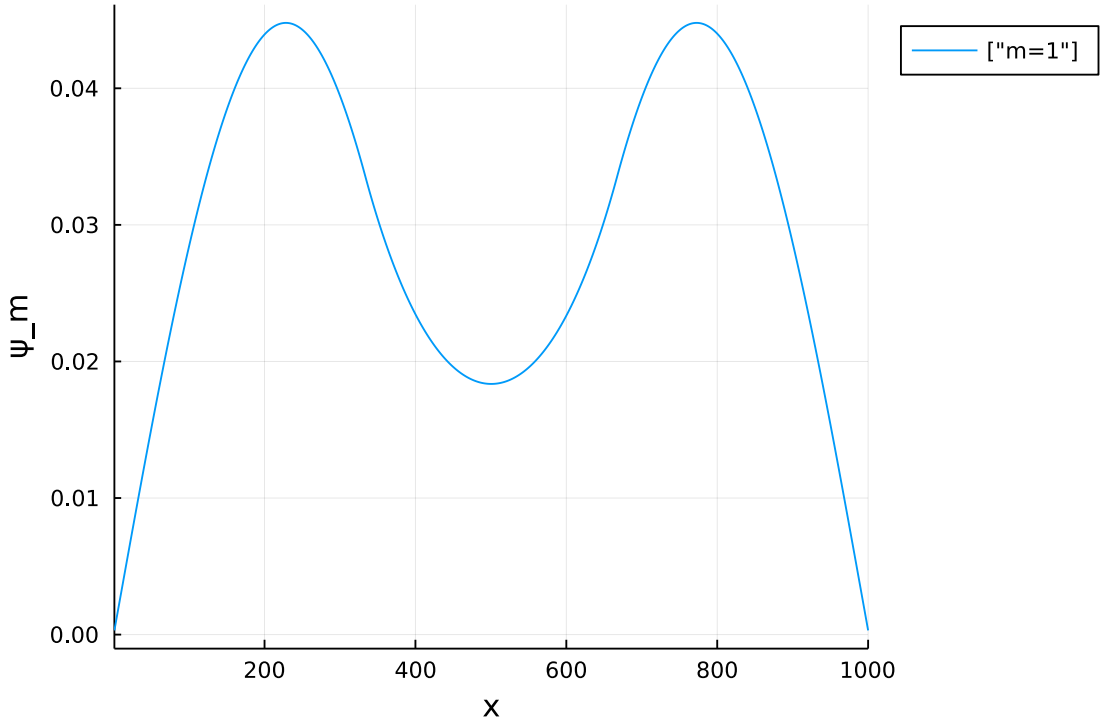
[12]:
```
_well, _well = eigen(H_well)
```

[12]:
```
Eigen{Float64, Float64, Matrix{Float64}, Vector{Float64}}
values:
1000-element Vector{Float64}:
     0.47327037462975785
     0.5298760753799698
     1.4026587505435573
     1.9586114754214419
     2.752084146516932
     3.927536095123729
     5.19556922878528
     6.605472404637603
     8.332454664869653
```

```
    10.209183649765777
    12.226875059796424
    14.527380442842874
    16.99848938095766

 39986.139762687344
 39988.39307775864
 39990.51003442546
 39992.3356447574
 39993.999364211515
 39995.54046175298
 39996.745723927605
 39997.82882070784
 39998.79340374444
 39999.325273624665
 39999.90541423738
 40000.66859605647
vectors:
1000×1000 Matrix{Float64}:
 0.000308129   0.000346155  -0.000417897  …  -0.000169873  -3.6298e-5
 0.000616243   0.000692291  -0.000835736      0.000339743   7.25985e-5
 0.000924328   0.00103839   -0.00125346      -0.000509611  -0.000108904
 0.00123237    0.00138443   -0.001671         0.000679474   0.000145216
 0.00154035    0.00173041   -0.00208831      -0.000849331  -0.000181539
 0.00184826    0.00207629   -0.00250533   …   0.00101918    0.000217873
 0.00215608    0.00242205   -0.002922        -0.00118902   -0.000254222
 0.0024638     0.0027677    -0.00333826       0.00135885    0.000290588
 0.00277141    0.00311319   -0.00375405      -0.00152866   -0.000326973
 0.00307888    0.00345852   -0.00416931       0.00169846    0.000363381
 0.00338621    0.00380367   -0.00458399   …  -0.00186824   -0.000399812
 0.00369337    0.00414861   -0.00499802       0.00203801    0.000436271
 0.00400037    0.00449334   -0.00541135      -0.00220776   -0.000472758

 0.00369337   -0.00414861   -0.00499802       0.00203801   -0.000436271
 0.00338621   -0.00380367   -0.00458399      -0.00186824    0.000399812
 0.00307888   -0.00345852   -0.00416931   …   0.00169846   -0.000363381
 0.00277141   -0.00311319   -0.00375405      -0.00152866    0.000326973
 0.0024638    -0.0027677    -0.00333826       0.00135885   -0.000290588
 0.00215608   -0.00242205   -0.002922        -0.00118902    0.000254222
 0.00184826   -0.00207629   -0.00250533       0.00101918   -0.000217873
 0.00154035   -0.00173041   -0.00208831   …  -0.000849331   0.000181539
 0.00123237   -0.00138443   -0.001671         0.000679474  -0.000145216
 0.000924328  -0.00103839   -0.00125346      -0.000509611   0.000108904
 0.000616243  -0.000692291  -0.000835736      0.000339743  -7.25985e-5
 0.000308129  -0.000346155  -0.000417897     -0.000169873   3.6298e-5
```

[13]:
```
plot( _well[1:N, 1:1], xaxis=("x", (1,N)), yaxis=(" _m"), label=["m=1"], legend=:
↪outertopright)
```

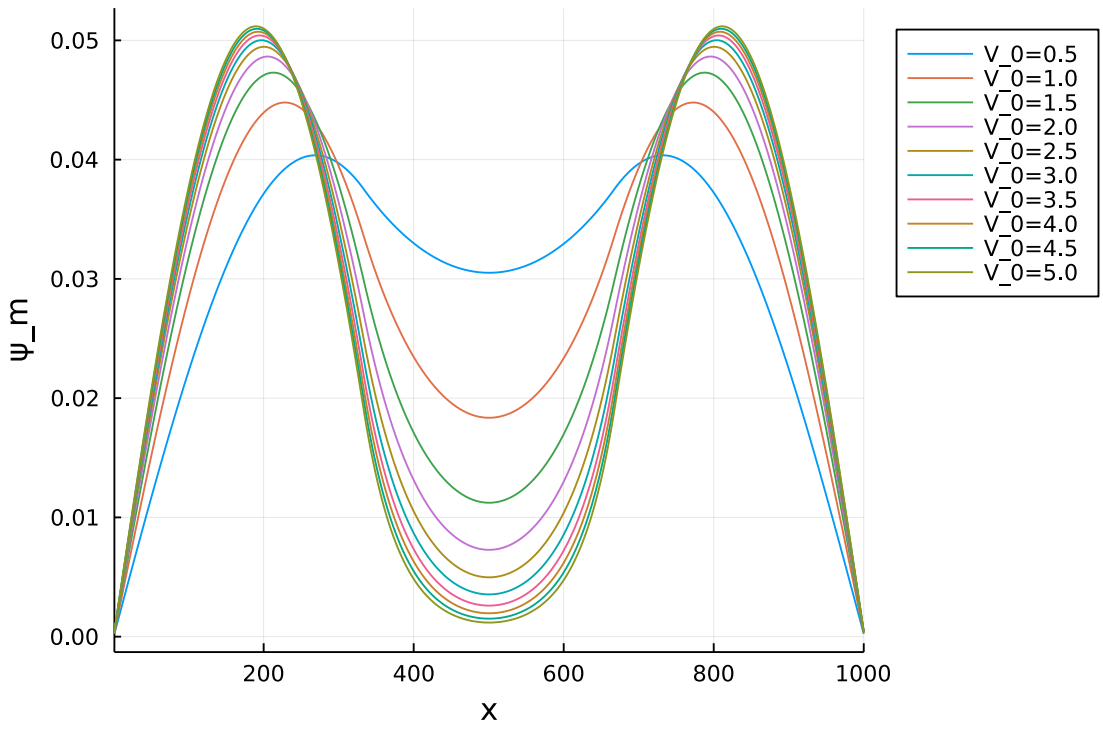
[13]:



Let us examine what difference the magnitude of the well potential V_0 make.

[14]:
```
function gs1(N,a)
    groundstates = []
    labels = Array{String}(undef, 1, 10)
    for v in 1:10
        V0 = v * 0.5
        mat_H = make_H1dv(N, a, V0)
         ,   = eigen(mat_H)
        push!(groundstates,  [:, 1])
        labels[v] = "V_0="* string(V0)
    end
    reshape(labels, 1, length(labels))
    return groundstates, labels
end
groundstates, labels = gs1(N, a)
plot(groundstates, xaxis=("x", (1, N)), yaxis=(" _m"), label=labels, legend=:
↪outertopright)
```

[14]:


0.05
0.04
0.03
0.02
0.01
0.00
ψ_m
200
400
600
800
1000
x
V_0=0.5
V_0=1.0
V_0=1.5
V_0=2.0
V_0=2.5
V_0=3.0
V_0=3.5
V_0=4.0
V_0=4.5
V_0=5.0